

MUSIC GENRE CLASSIFICATION

Solomon Asad

Student# 1008295890

`solomon.asad@mail.utoronto.ca`

Sujit Peramanu

Student# 1007812734

`sujit.peramanu@mail.utoronto.ca`

Gurpreet Singh Mukker

Student# 1005985971

`gurpreet.mukker@mail.utoronto.ca`

Aidan Dempster

Student# 10050827044

`aidan.dempster@mail.utoronto.ca`

ABSTRACT

Music has become an integral part of life because of its ability to uplift emotions and connect people in social settings. Music can be divided into genres in numerous ways, sometimes broadly and with polarity, such as popular music, in contrast to art music or folk music as an example. The proliferation of popular music in the 20th century has further led to over 1,200 definable subgenres of music. With such classifications, this project aims to develop a robust deep-learning model capable of classifying music into specific genres based on audio samples. Automated genre classification has become an essential aspect of music recommendation systems on applications such as Spotify, digital libraries, and audio content management software. As such, we use a convolutional embedding model trained with the InfoNCE contrastive loss on the large MTG audio dataset (Bogdanov et al., 2019). We then use the logits from this pre-trained network as features for downstream classifier models trained on the GTZAN genre classification dataset (Tzanetakis et al., 2001). We evaluate both a single layer feed forward neural network and a random forest. We also evaluate the performance of a random forest classifier on features derived using classical audio processing methods. We find that both deep methods achieve approximately 76% accuracy while the classical method achieves 50% accuracy. On a newly collected dataset the deep method achieves an accuracy of 56% while the classical baseline has an accuracy of 31%.

—Total Pages: 9

1 INTRODUCTION

The goal of the project is to develop a deep-learning model that can accurately classify music tracks into genres based on short audio samples.

The motivation behind this project stems from the challenges associated with manual music classification, which is both time-consuming and inconsistent due to low inter-classifier consistency. Using a deep learning model with high accuracy for this process enables more personalized music recommendations and efficient music discovery, which are incremental for user engagement and satisfaction in streaming services. Furthermore, this project can also support various applications in musicology, such as studying genre evolution and regional music trends.

Convolutional Neural Networks, for example, are known for their prowess in image and audio processes, and are appropriate for analyzing time-frequency representations of audio signals such as spectrograms.

These networks can learn hierarchical representations, making them effective at recognizing the spectral and temporal patterns indicating different music genres.

Unfortunately, due to copyright laws, many genre prediction datasets are small, making large models trained on them prone to overfitting. However, there is a multitude of unlabeled or weakly labeled audio on the internet. We propose the use of two dataset, one large and unlabeled and one small and labeled. We perform representation learning as a pre-training step to learn transformations of the data that promote clustering of music with similar structures. Since music is inherently a time series, a natural contrastive objective is to maximize the pair similarity between embeddings of two sections of the same piece of music as shown in Figure 1.

2 ILLUSTRATION

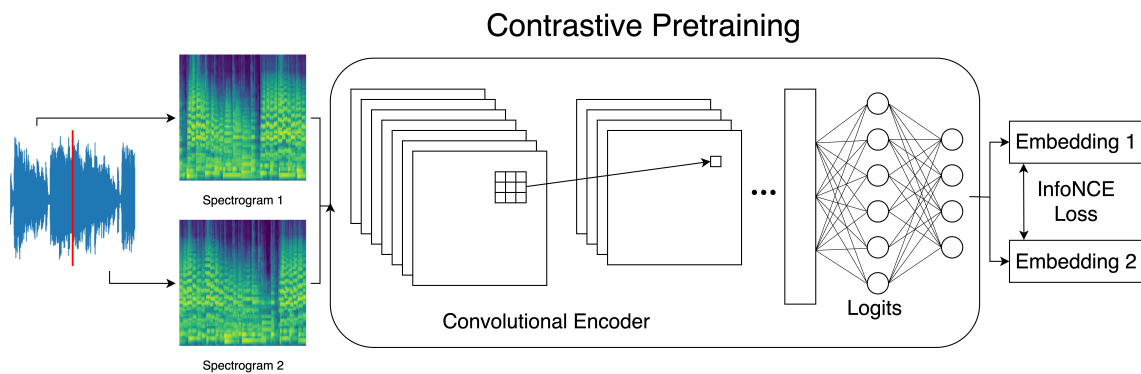


Figure 1: During pre-training, we split a waveform in half, convert each to a mel spectrogram, pass them through the encoder, and maximize similarity between the embeddings.

After training, the logits from the contrastive model contain semantic information about the contents of the input audio. This means models that take these logits as input do not need to be large in order to perform complex tasks like genre prediction as demonstrated in Figure 2.

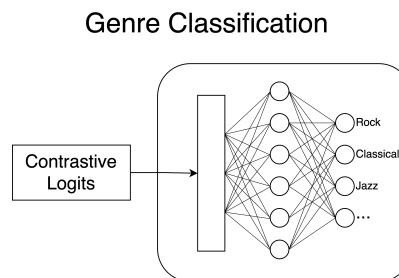


Figure 2: Single layer MLP genre classifier using contrastive logits as input.

3 BACKGROUND AND RELATED WORK

The field of audio machine learning is deeply rooted in signal processing, which involves analyzing, modifying, and synthesizing signals such as sound. Key concepts include Fourier transforms, filtering, and convolution.

Early research in signal processing laid the groundwork for modern audio analysis. The Fast Fourier Transform (FFT) development in 1965 by Cooley and Tukey revolutionized the field by enabling efficient computation of the Fourier transform.

Below are 5 descriptions of papers highly linked with the field of audio machine learning.

3.1 SPEECH EMOTION RECOGNITION FROM 3D LOG-MEL SPECTROGRAMS WITH DEEP LEARNING NETWORK

This paper focuses on enhancing speech emotion recognition (SER) using a novel architecture ADRNN, which incorporates dilated CNNs with residual blocks and BiLSTM based on the attention mechanism. The study focuses on 3D Log-Mel Spectrograms to improve feature extraction, crucial for SER performance. The paper demonstrates improvements in classification accuracy compared to previous methods and allows for effective learning of long-term dependencies. (Meng et al., 2019)

3.2 CNN ARCHITECTURES FOR LARGE-SCALE AUDIO CLASSIFICATION

In this paper researchers used a massive YouTube dataset to evaluate deep neural networks for audio classification, specifically soundtrack labeling. They explored various architectures, including Inception-V3 and ResNet-50, adapted from image classification. The large dataset enabled testing high-capacity models and assessing the impact of training set size on performance. Additionally, they investigated the models' usefulness for acoustic event detection. (Hershey et al., 2017)

3.3 VOICE BASED SERVICES USING AUDIO CLASSIFICATION

The authors utilized the Speech Commands dataset, comprising 65,000 one-second audio recordings of 30 distinct English words. These audio recordings were preprocessed into spectrograms, visual representations of audio frequencies over time. These spectrograms served as input for a convolutional neural network (CNN). The CNN model was trained on the labeled spectrograms, enabling it to learn and identify patterns within the spectrograms that corresponded to specific words. This process allowed the CNN to classify new audio inputs based on the learned patterns, effectively performing audio classification. (Jay Prakash Malviya, 2022)

4 DATA PROCESSING

1. A lot of the data processing is shared between both the contrastive pre-training and the classifier training even though they use different datasets.
2. Since GTZAN and MTG dataset were sampled at different rates, MTG was downsampled from 44000Hz to GTZAN's sample rate of 22050Hz. Audio waveforms were then converted to Mel-spectrograms with following parameters.
Frequency Bins = 64 , Window Length = 1024, Hop Length = 512
3. Augmentation through frequency and time masking. This is useful due to the fact that it forces the network to learn to be robust to changes in the audio. Basically, it should recognize that two slices of audio are the same even when some noise is added.

4. Stratified splits. These ensure that the train, val, and test sets all have even numbers of samples from each class which ensures good performance in the training phase and valid accuracy measures in the val and test phases.

4.1 MTG

For contrastive pre-training we are using MTG. This is a large dataset, but the genre labels do not align with those that we want to classify. This makes it perfect for doing pre-training.

The full data loader for the MTG dataset is here. It takes in a path to the unzipped tars in the dataset and returns pairs of mel spectrograms. These pairs come from the same song and we ensure that no other samples in a batch come from that song. If they did, it would break the contrastive loss.

The most interesting thing about this dataloader is that the dataset is too large(53GB) to fit in memory all at once so it needs to incrementally load audio files while running. The audio files are also in .mp3 format which from our testing were very slow to load compared to same files but in a torch tensor format. Since mp3 loading is a CPU process, it throttled the training process even when using a powerful GPU. Hyperparameter tuning therefor became impossible.

So separate script was written that used multiple processes to convert all the mp3 files into melspectrograms first. These spectrograms were then fed directly into model for training. This reduced the single session training time of 25 epochs from 24 hours to about 5 hours.

To make the loading process even faster, we used multiple workers(multiple processes) in the Pytorch Dataloader class that allowed the loading of multiple batches in parallel to match the speed at which GPU was processing them in forward and backward passes.

The dataset samples a fixed number of windows for each audio file in order to not put too much focus on any one type of music. For example, if classical pieces are generally 40 minutes and pop songs are 2 minutes, we do not want the network to have 20 times more samples from a classical piece than a pop song. Therefore, taking n samples from each track is more fair.

In order for the contrastive assumption, that for each sample in a batch there is only one positive sample, to be true, the dataset needs to be able to return samples from $[batch_size]$ different tracks together. To achieve this, we load $[batch_size]$ tracks into memory and iterate over them all in parallel. This ensures that in each batch, samples from each song always come in pairs.

We use the first 31 tars in the MTG dataset which comes out to 16343 tracks. For each track we take 10 samples so the total number of samples in the dataset is 163430.

4.2 GTZAN

For the genre prediction, we are using the classic GTZAN dataset. This has 1000 tracks with 100 in each genre. The genres are blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock.

We use the same masking technique for augmentation. It is more important here because the dataset is smaller.

The number of samples is difficult to quantify because we randomly sample a different portion of each track on each epoch. Since the dataset is small, this helps with preventing memorization of specific sections of the tracks. If we count the number of samples as the number of non-overlapping samples that can be taken from the dataset, it is of size 3000 since each track is 30 seconds and we take 10 second samples.

5 BASELINE MODEL

We use a random forest applied to a set of classical features for audio processing. We use zero crossing rate, spectral centroid, spectral rolloff, spectral contrast, and spectral bandwidth. Each of these outputs a time series so we take the mean and standard deviation of each to use as inputs to the random forest.

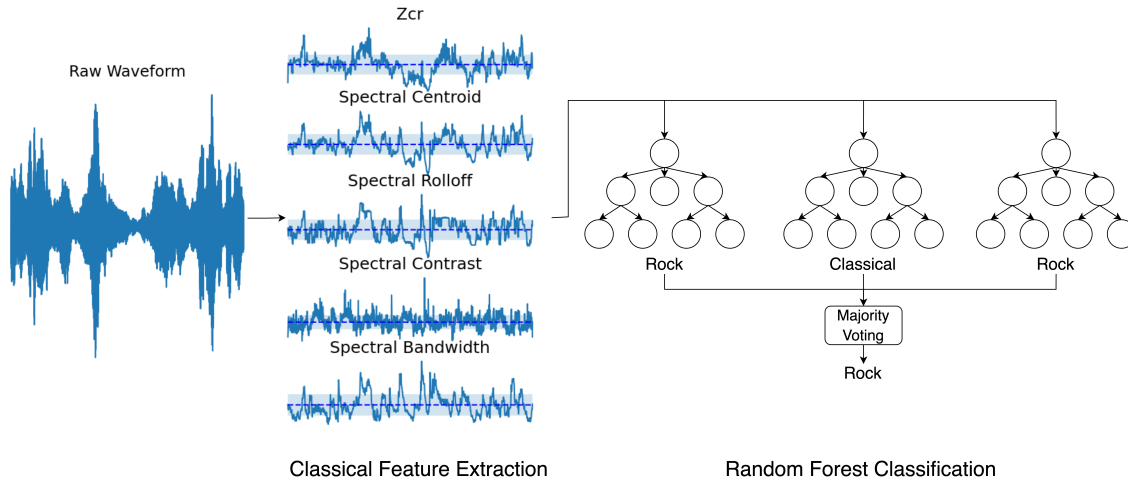


Figure 3: Random Forrest Classification

We tuned the number of trees based on performance on the validation set.

This model was easy to implement and train. LibRosa allows us to easily compute the features and scikit-learn implements a random forest classifier which can be trained by passing in features and labels as numpy arrays.

6 ARCHITECTURE OF PRIMARY MODEL

The primary model consists of two components.

1. The contrastive model that we pre-train.
2. The classification head that may either be an MLP or a random forest.

6.1 CONTRASTIVE PRE-TRAINING

The backbone of the contrastive model is a basic 5 layer CNN with a constant kernel size of 3×3 , ReLU activation, and 2×2 max pooling after each. The number of filters doubles after each layer, being 32, 64, 128, 256, and finally 512. After the final convolution we flatten and map to 1024 dimensions. Since this is a contrastive model, we also add a final layer to project into a lower dimensional contrastive space where the loss will be computed. We choose a 128 dimensional contrastive space to align with previous works such as the smallest CLIP model (Radford et al., 2021)

Following the standard set by CLIP, we use the InfoNCE loss to train our contrastive model. The InfoNCE loss extends the idea of categorical cross entropy to distributions of non-fixed size. In cross entropy the

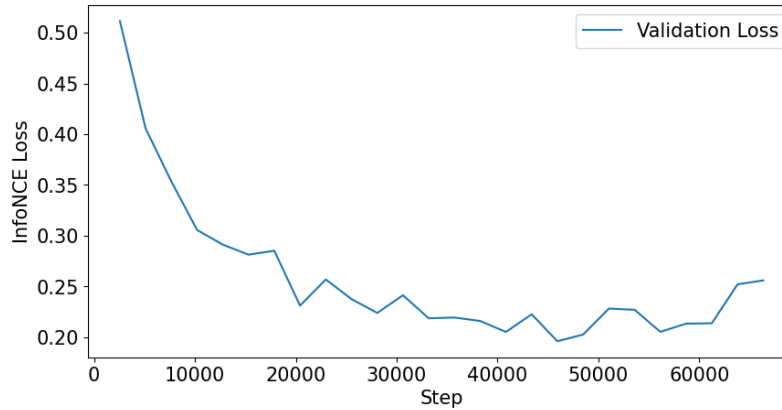


Figure 4: InfoNCE loss curve

model outputs logits that are then passed through a softmax to convert them to a probability distribution that cross entropy acts on. In InfoNCE, we instead embed n pairs of samples. For each of these pairs we compute a separate cross entropy loss. Instead of directly predicting the logits, we define the logits as a dot product between normalized embeddings. In mathematical terms

$$\text{InfoNCE}_a = -\log \left(\frac{\exp(e_a \cdot e_p)}{\sum_i \exp(e_a \cdot e_i)} \right)$$

Here e_a and e_p form the pair of samples drawn from the same audio while the rest of the samples are used to form a probability distribution. By minimizing this loss, we by proxy maximize the dot product between the embeddings that were drawn in pairs and minimize the dot product between all other embeddings.

In Figure 4 we see some signs of overfitting at the end of training. This implies that we could improve performance further by expanding the dataset or enhancing augmentation. However, this training procedure is already computationally expensive and we would require improved hardware to train this model efficiently.

After the model is trained, we can freeze the backbone and use it to extract semantic features from audio signals. For basic analysis, we plot the embeddings in a 2D by using TSNE and visualize how the genres cluster in the logit space of the contrastive model.

Figure gives another insight into why the baseline model performed well on classical and poorly on rock. The contrastive model was only trained to differentiate between tracks, not directly to predict genre. However, we do see clusters which implies that some genres are similar enough that different tracks in the genre could be mistaken for each other. Classical forms a tight cluster at the top while rock forms small clusters throughout the embedding space. This implies that the tracks labeled "rock" in the dataset do not share much semantic similarity so it may have been more of a grab bag of many types of music which may be similar to other genres.

6.2 GENRE CLASSIFICATION USING CONTRASTIVE LOGITS

The 1024 dimensional logits of the contrastive model should have very dense information content so we are free to choose whatever simple classifier we want to predict genre.

We evaluated both a random forest classifier and a single feed forward layer with a softmax activation. Using the validation set, we chose the feed forward layer as our final classifier.

7 QUANTITATIVE RESULTS

To gather results of our model’s performance we created a test set when the GTZAN dataset was first loaded which was never seen before by the models nor used to tune the parameters. These were the efforts taken to ensure the results were a good representation of the model’s performance on new data. We evaluated the model on the new data through the accuracy as the main metric for each of the models on this test set. First, we evaluated the end-to-end CNN that was trained on GTZAN.

As seen in figure 5, on the base GTZan dataset, our baseline random forest classifier applied on classical audio features achieved a test set accuracy of 50.1% while the classifier trained on the logits of our contrastive model achieved a test set accuracy of 76.3%. Both outperform random guessing which is 10% since we have 10 balanced classes. We also train a model with the same architecture as our contrastive classifier without contrastive pretraining which achieves an accuracy of 76.6% on the test set.

On the expanded GTZan dataset that we collected ourselves, we see an interesting effect. All accuracies decrease, but the model that used contrastive pre-training had a reduction far less pronounced than the model trained end-to-end on the base GTZan.

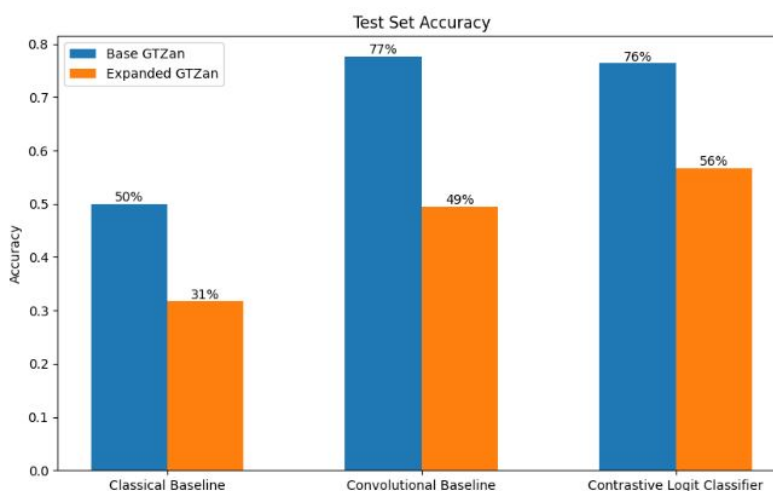


Figure 5: Test Accuracy’s for Different Test Sets

8 QUALITATIVE RESULTS

In figure 6 we see that the contrastive classifier outperforms the baseline classifier in all genres. We see a general trend that blues and rock are difficult to classify across all three evaluations and that classical is the easiest to classify.

In figure 7 we use UMAP to visualize clusters in the 1024 dimensional logit space in 2 dimensions (McInnes et al.). This allows us to intuitively understand why some classes may be easier to differentiate than others and which classes have the largest internal diversity.

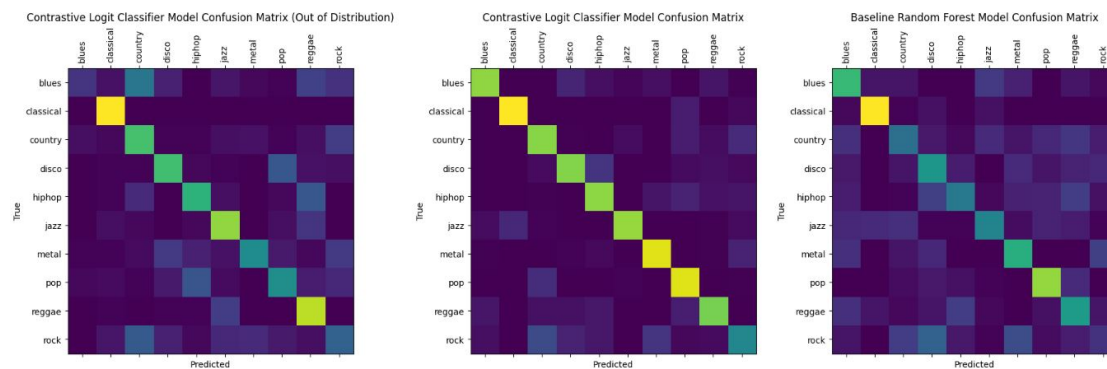


Figure 6: Left - Confusion matrix for the baseline random forest model. Middle - Confusion matrix for the contrastive classifier applied to the base GTZan dataset. Right - Confusion matrix for the contrastive classifier applied to the expanded GTZan dataset.

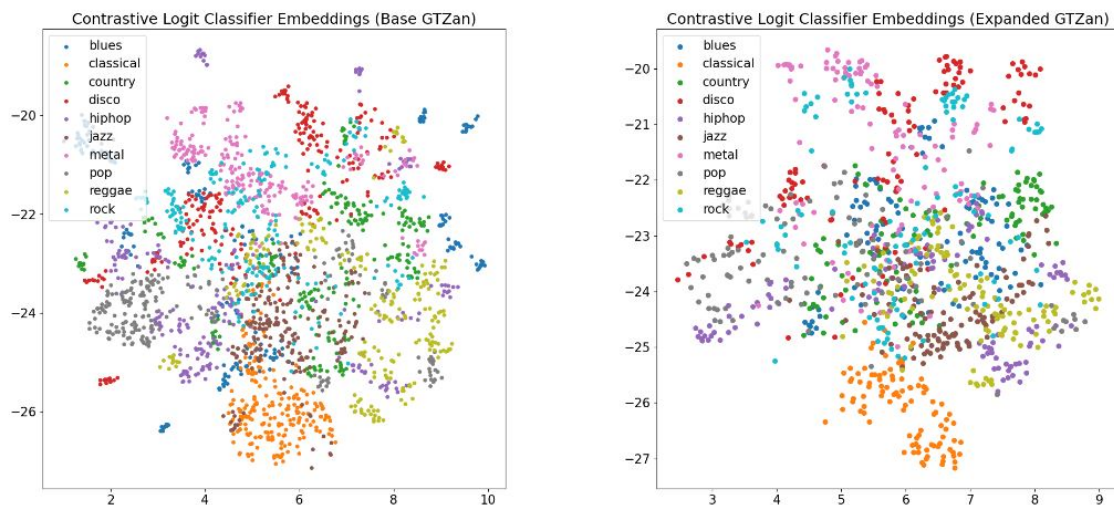


Figure 7: Embeddings from the pre-trained contrastive backbone from the base GTZan dataset, left, and the expanded GTZan dataset, right visualized using a UMAP dimensionality reduction (McInnes et al.)

9 DISCUSSION

Since all classification models were trained using data from the base GTZan dataset, the fact that figure 5 shows that the model that takes as input contrastive logits loses less accuracy when evaluating on a never seen before dataset shows the benefit of pre-training lies in creating models that are more robust to distribution shifts. The contrastive backbone has learned a transformation of the data that pulls semantically similar music together in the embedding space meaning that even though none of the music from the expanded GTZan set was seen during training, the similarity to music that was seen during training can be exploited by the classifier.

Figure 7 gives us more insight into why certain classes were easier or harder to classify. Since the contrastive model was only trained to bring embeddings from the same song together, any broad clusters of many songs reflect shared structures across the entire genre. In figure 6 we see that classical was well classified by all models. With the contrastive embeddings, it is clear this is because all classical music shares similar features and so forms a single tight cluster. In contrast, the rock genre forms small clusters throughout the embedding space, often near the genres of pop, disco, and metal which most people would agree share semantic similarities. The case of blues is more interesting. In the base GTZan set embeddings, we can see multiple small clusters, but each cluster is separated from each other. Since blues has long periods of vocal or instrumental only, it consists of many semantically dissimilar sections which explains why it is also difficult to classify.

In total, our results do not match the state of the art which nears 100%. However, since we restricted ourselves to a small labeled dataset and relatively small models, our results are comparatively competitive. We also took on the challenge of using training techniques and data loading methods not covered in any labs which likely reduced the final accuracy, but allowed us to learn about methods we otherwise would not have. Contrastive pre-training allowed us to see the generalization benefits of an unsupervised learning strategy and the data-hungry nature of the technique forced us to learn how to load datasets that cannot fit in ram and are expensive to process on the fly.

10 ETHICAL CONSIDERATIONS

When using the MTG-Jamendo dataset for music genre prediction, it's important to respect the Creative Commons license and copyright regulations. Since our work is for educational purposes, it therefore qualifies as fair use. Due to genre imbalance in the dataset, the final model might be biased towards overrepresented genres, necessitating careful data curation.

Cultural sensitivity is also of prime importance; the model must accurately classify diverse musical styles without perpetuating stereotypes or misinterpreting cultural nuances. Therefore we must be aware of potential discriminatory outcomes and avoid using the model in applications where the primary users are not represented well in the data.

11 GITHUB & GOOGLE DRIVE

- <https://github.com/Veldrovive/aps360-audio>
- https://drive.google.com/drive/folders/1gqYGBAECBNKzyEfkcmb7Uh9Otz_X82I1?usp=sharing

REFERENCES

- Dmitry Bogdanov, Minz Won, Philip Tovstogan, Alastair Porter, and Xavier Serra. The mtg-jamendo dataset for automatic music tagging. In *Machine Learning for Music Discovery Workshop, International Conference on Machine Learning (ICML 2019)*, Long Beach, CA, United States, 2019. URL <http://hdl.handle.net/10230/42015>.
- Shawn Hershey, Sourish Chaudhuri, Daniel P. W. Ellis, Jort F. Gemmeke, Aren Jansen, R. Channing Moore, Manoj Plakal, Devin Platt, Rif A. Saurous, Bryan Seybold, Malcolm Slaney, Ron J. Weiss, and Kevin Wilson. Cnn architectures for large-scale audio classification. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 131–135, March 2017. doi: 10.1109/ICASSP.2017.7952132.
- Dr. Shamshekhar S. Patil Jay Prakash Malviya. Voice based services using audio classification. *International Journal Creative Research Thoughts*, 10:282–288, 2022.
- Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. URL <https://arxiv.org/abs/1802.03426>. Version Number: 3.
- Hao Meng, Tianhao Yan, Fei Yuan, and Hongwei Wei. Speech emotion recognition from 3d log-mel spectrograms with deep learning network. *IEEE Access*, 7:125868–125881, 2019. doi: 10.1109/ACCESS.2019.2938007.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. 2021.
- George Tzanetakis, Georg Essl, and Perry Cook. Automatic musical genre classification of audio signals, 2001. URL <http://ismir2001.ismir.net/pdf/tzanetakis.pdf>.